Relation Ownership and Bag Preserving Transfer

Mettapedia Working Notes

1 November 2026

Abstract

Eighteen Lean modules establish independent semantic contracts for relation-column ownership, retain/release receipts, dead-ID reuse and row take-over. The bookkeeping algorithm updates each column occurrence while maintaining one external hold per used ID. Transfer preserves the full occurrence bag, even when duplicate copies remain as rows or the target gives its representation back during the pass. Publication composes this bag law with the existing captured-reader protocol. Separate root owners expose descriptor and snapshot retention, and local bridges connect semantic movement and zero-crossing counts to existing cost models. Stable renumbering preserves decoded rows, copied use counts and side-index candidates; original logical coordinates connect physical removal marks to the bag law. A UInt32 realization preserves counts and receipts under a whole-row capacity condition. Unique binary relations inherit a stronger bound from their finite numbering; general tuple tables do not. Native pointer updates, enforcement of the range conditions, root discovery and publication isolation remain implementation obligations. An incremental publication worker preserves its compensated bag at every private step. Snapshot root enumeration and a separate native-metadata lease model compose term collection with descriptor retirement. Pinned-executable allocation-failure controls distinguish recovery, fatal termination and a deliberately broken publication step. An additional matching-admission law exposes a false absence certificate for open compact facts in the pinned runtime; the expanded native gate is not green.

1 Column multiplicity and external ownership

Let R be the list of live relation entries, each entry a list of stored IDs. The reference demand for ID i is

$$D_R(i) = \sum_{r \in R} \mathrm{count}_i(r).$$

This is the existing `ArenaHeldResources.logHolds` interpretation, reused without redefining it to be the target bookkeeping algorithm.

`RelationColumnOwnership` stores a counter function u and a finite external-ownership set H. Its invariant is

$$u(i) = D_R(i), \qquad i \in H \iff u(i) > 0.$$

Insertion walks the columns, increments each counter and takes a hold only when the previous counter was zero. Removal walks them in the same manner, decrements and releases only when the previous counter was one. The proofs establish the invariant for insertion and membership-checked removal.

A represented row supplies enough demand for every decrement, including repeated occurrences of the same ID. The safe-removal theorem proves this at each intermediate step, not just for the final subtraction. For example, the row $[7, 7]$ contributes two internal uses and one external hold.

Removing it releases the hold only after the second decrement. If another row $[7, 9]$ remains, ID 7 stays held.

The unique-insert wrapper refuses a row already represented in the dense relation. This is separate from the surrounding space's bag semantics: additional copies can remain in plain rows. A presence bit is therefore neither a column-use count nor a complete occurrence count.

2 Callbacks are occurrences too

An external reference counter observes callback invocations. Equality of owned sets alone would not detect two redundant retain callbacks. `RelationHoldReceipts` executes the zero-crossing receipt algorithm and proves that its acquired and released ID lists have no duplicates. Membership is characterized exactly:

$$i \in \mathsf{acquired}(u, r) \iff i \in r \ \wedge \ u(i) = 0,$$
$$i \in \mathsf{released}(u, r) \iff i \in r \ \wedge \ u(i) = \mathrm{count}_i(r),$$

where the release theorem requires sufficient initial counts. These lists are sublists of the input columns. They consequently emit exactly one callback for each required ownership transition and at most one per column.

The ownership-change theorems identify acquisition with newly owned IDs and release with IDs no longer owned afterward. The controls reject a naive per-column callback sequence $[7, 7]$ for a newly inserted repeated-column row: the correct retain receipt is $[7]$.

The model uses natural counters. Both inspected C stores use `uint32_t holds[]` and unchecked increment/decrement operations. The logical safe-removal result supplies a no-underflow premise when the native representation invariant holds. The following finite-word results make the overflow condition precise; they do not demonstrate an allocated overflowing runtime workload.

3 Finite words and relation-specific capacity

`RelationWordOwnership` implements the updates with Lean's actual `UInt32` arithmetic: hold tests the old word and increments it; unhold decrements first and tests the new word. Interpretation takes each word's natural value and retains the ownership set. For an inserted row r, the necessary range premise used by the refinement is

$$\forall i, \quad u(i) + \mathrm{count}_i(r) < 2^{32}.$$

Under that premise, the complete word update equals the natural-count update, including the ordered acquire receipt. Removal of a represented row inherits the earlier no-underflow proof and preserves the release receipt. The resulting represented store remains valid.

The executable preflight counts all occurrences of an ID in the row before performing any update. A second version compares the representable addition against `UINT32_MAX - old`; it is proved equivalent to the natural condition and does not evaluate an overflowing sum. Preparation refusal publishes no updated counter state. The inspected C code does not currently perform this preflight; the result is a permission law for such admission.

Checking room for one use against the old counter is insufficient for a row containing an ID twice. A control starts ID 7 at $2^{32} - 2$ and inserts $[7, 7, 9]$: unchecked increments wrap 7's count to zero while its ownership set still contains 7. The checked preflight refuses that row but accepts $[7, 9]$. Decrementing zero likewise wraps, rather than acting as truncated natural subtraction.

`RelationOccurrenceBounds` bounds every ID's count by total column volume and then by arity times fact count. This is sufficient but not always sharp: different IDs can each fit while their combined volume exceeds one counter's range.

For unique binary pairs, `RelationBinaryUseBound` proves a stronger law. With n numbered IDs, a fixed ID has at most n distinct partners in each column, so its use count is at most $2n$, independent of the number of pairs. The numbering constraints

$$4n \le 3s, \qquad s \le 2^{31}$$

imply $n < 2^{31}$. A fresh pair therefore satisfies the whole-row word admission condition. This licenses omission of a separate counter preflight *if* the native representation maintains pair uniqueness, correct counts and those numbering bounds. A repeated-pair bag fails the bound, which is why extra bag copies staying outside the unique relation matters.

Higher arities do not inherit that argument. The checked symbolic family

$$[x, x, x, x, x, x, x, y_j], \qquad 0 \le j < 613\,566\,757,$$

has distinct eight-column rows and at most 613 566 758 IDs. Both its row count and ID count fit a three-quarter-loaded 2^{30}-slot table, but x has $4\,294\,967\,299 > 2^{32}$ column uses. The proof instantiates general list/count laws; it does not allocate or enumerate that runtime workload. It shows that uniqueness and these hash-load bounds alone cannot discharge the tuple counter's range obligation. A further control compares release receipts after that count is encoded in a word: removing seven uses of x and the sole use of another ID releases both in the wrapped worker, whereas the natural-count worker releases only the other ID. Overflow can therefore change ownership observations, not merely a private numerical statistic.

4 Dead dictionary entries and identifier reuse

An unused number may remain in a dictionary without retaining the term it once named. `RelationIdReuse.reserve` reuses an existing raw-ID entry or appends a new one. It proves that the returned index names the requested ID, old indices are preserved, and distinct dictionaries stay distinct. The state, dictionary and lookup share one admitted identifier namespace; cross-universe reuse or reset requires its own revalidation contract.

The dictionary contains IDs, not cached payload meanings. Reading through a reserved number consults the current lookup. Thus a dead entry naming raw ID 7 can correctly name a newly allocated payload after that ID is reused. A retained old-payload cache would be wrong. Missing lookups remain explicit `none` values in the denotation; they are not filtered into apparent absence of a fact.

The column invariant proves that a zero-use ID occurs in no live relation row. Changing its meaning therefore preserves that relation's existing rows. This is not yet global permission to reuse it: another relation, a caller, a descriptor or a captured reader may still protect it. The checked reuse operation requires absence from a supplied complete protected set and preserves every lookup in that set.

Positive controls remove the old repeated-column row, retain its dead number, change the raw ID's meaning and reinsert a row that observes the new payload. Negative controls show both stale-payload reuse and a stale side entry inventing a fact. Correct native side indexes must describe the same live entries as the counters; the dictionary theorem alone does not establish that array/index correspondence. The next section supplies the finite-list renumbering and candidate-index part of the obligation.

5 Stable renumbering and side-index candidates

For an old dictionary D and live mask b, define

$$\rho_b(i) = \mathrm{count}_{\mathsf{true}}(\mathrm{take}_i\, b).$$

`StableNumberCompaction.pack` scans the mask and dictionary together, retaining live entries in their original order. With equal mask and dictionary lengths, a live coordinate satisfies

$$\mathsf{pack}(b, D)[\rho_b(i)] = D[i].$$

Lookups are optional, so invalid coordinates are not silently replaced by a default value. The map is strictly increasing on live coordinates, satisfies $\rho_b(i) \le i$, and covers every compact coordinate without gaps. Sorted side arrays therefore remain sorted. For bit rows the destination word is never later than the source word. That last bound is a necessary condition for the inspected in-place loop, not a complete proof of its mutable writes: the current source word must still be captured before it is cleared.

`RelationNumberingCorrespondence` independently decodes numbered rows through D. It builds b from positive column-occurrence counts, packs D and maps every row coordinate through ρ_b. For a valid dictionary and in-range columns, the entire decoded row sequence is unchanged, the output is valid and every retained number is used. The compact dictionary contains exactly the IDs named by live columns.

The copied hold-counter array also agrees with the relocated row counts. For live i,

$$D_{\mathsf{rows}_{\mathrm{after}}}(\rho_b(i)) = D_{\mathsf{rows}_{\mathrm{before}}}(i).$$

This uses injectivity on live coordinates; it is not valid for arbitrary dead coordinates. A checked control shows a dead coordinate's rank aliasing a live one, which is why dead side entries cannot be relocated as facts.

An exact side-query law preserves the original row positions selected by a column and key, after relocating the key and columns together. Repeated equal rows retain separate positions. This specifies the required content of rebuilt hash tables and side chains without claiming their C mutation code has been verified. A control with dictionary $[8, 7, 6, 9]$ and rows $[[1, 1], [3, 1], [1, 1]]$ compacts to dictionary $[7, 9]$, rows $[[0, 0], [1, 0], [0, 0]]$ and hold counts $[5, 1]$.

6 The roots have different owners

The inspected runtime separately retains a kept relation's head and the constructor IDs in its shared skeleton. Removing its last fact column does not automatically retire that descriptor. Accordingly, `RelationRootOwners` separates

$$H_{\mathrm{columns}}, \quad H_{\mathrm{descriptor}}, \quad H_{\mathrm{captured}}.$$

Their union is the direct root support of the declared owners. It is embedded in the existing `ResourceOwnership` finite-heap model, whose reachable footprint includes shared and cyclic descendants. Reuse is constrained by that complete footprint rather than merely by the direct roots.

Under root-allocation validity, every live column resolves to a payload. Protected reuse preserves live relation denotation, descriptor reads and captured-owner reads. A separate bridge derives the captured roots from the existing reader's saved row view and proves preservation of the values read from its remaining suffix.

These root sets describe reachability support, not callback multiplicity. The exact callback accounting is the separate receipt result above. Nor does this ownership split prove that retaining an empty descriptor is the best cache policy. A stronger policy that no metadata IDs remain after the last fact requires retirement or reconstruction of that descriptor. The current column theorem must not be advertised as proving that stronger property.

6.1 A snapshot needs both term roots and live metadata

`RelationCapturedRoots` models the inspected callback: enumerate the head, the constructor IDs of non-hole skeleton nodes, all declared columns, and every materialized record-cache ID. Its independent reader observes the complete supplied fields. Under the layout equations

$$|\mathsf{columns}| = \mathsf{count} \cdot \mathsf{arity}, \qquad |\mathsf{records}| = \mathsf{count},$$

the callback list contains exactly the IDs required by that reader. Collection retaining these roots preserves every field and every reachable path. Filling an empty record-cache slot preserves the layout and adds the record's ID to the next enumeration. A root list captured before that fill is not sufficient for a later collection.

The callback itself also reads native memory. `RelationSnapshotMetadata` separates the native-address heap from the term-ID heap. Its loader requires the snapshot allocation, its owning universe and its separately allocated skeleton. Keeping every term alive does not reconstruct a freed skeleton or universe object. A prepared census checks the universe address of every provider and refuses the entire census if any provider fails to load; it never turns a failed provider into an empty root list. Completeness here is relative to the supplied provider registry. Discovering every native owner remains a separate obligation.

The model reuses `ResourceOwnership.AcquiredUse`: detaching a descriptor's current slot leaves an already admitted reader's lease valid. The final reader can extract the retired descriptor exactly once. The composition theorem preserves the full snapshot readout after descriptor detachment, native-metadata collection and term collection. It retains outside roots in both heaps. The theorem requires valid layouts, allocated term roots, the same admitted universe and live metadata dependencies; native pointer access, provider synchronization and universe-reset protocols must realize those conditions.

Controls distinguish a missing cache entry from a dangling materialized record, reject a foreign-universe provider, and show why keeping only column IDs is insufficient. Another control frees the skeleton while preserving all term lookups: the metadata loader correctly fails.

6.2 The provider registry has a semantic interpretation

`RelationProviderRegistry` reuses `RegistryIndexCost`'s existing slot array, coherent index, append and swap-removal operations. For a current callback enumeration E, its root census is

$$C(R, E) = \bigcup_{p < R.\mathsf{len}} E(R.\mathsf{slot}(p)).$$

Index coherence proves that this is exactly the union over registered keys. Adding key k changes the census to $E(k) \cup C(R, E)$; adding it twice does not add another registry slot. Removing k is equivalent to enumerating the old registry with only k's enumeration replaced by the empty set. It is *not* generally $C(R, E) \setminus E(k)$, because other providers may still name those IDs. The swap-removal proof retains the moved last slot's roots at its new coordinate.

Composing that census law with the captured-root theorem preserves a registered snapshot's readout across collection. The enumeration is evaluated at collection time; registration does not

cache its old result. A control adds a materialized root to a registered provider and shows that a stale census would miss it. The key carrier is natural numbers naming complete callback/context identities within one admitted universe. Encoding those identities and enrolling every external owner remain native obligations.

7 Transfer preserves the full bag

`RelationBagTransfer` models one target relation with unique dense entries and rows into which it may give its representation back. The source is an occurrence list. Each offer has one of four dispositions: refuse, store, give back and refuse, or store and then give back. An unavailable relation does not accept subsequent stores.

The pass records one Boolean decision per original source occurrence. A successful offer removes that occurrence from the source; a refusal retains it. Target give-back does not alter the original decision coordinates. For every action schedule,

$$[\mathsf{remaining}] + [\mathsf{dense}_{\mathrm{after}}] + [\mathsf{spilled}_{\mathrm{after}}] = [\mathsf{source}] + [\mathsf{dense}_{\mathrm{before}}] + [\mathsf{spilled}_{\mathrm{before}}],$$

where brackets denote multisets. This is a full bag equality, not equality of supports or total sizes. The dense list remains duplicate-free whenever it started so. Framing an unaffected bag preserves the equality, providing the local composition law for other relations.

For source rows $[7, 7, 9]$ and an empty target, storing every admissible row leaves $[7]$ as plain rows and dense entries $[9, 7]$. Deleting the refused second copy would lose an occurrence. If the first accepted entry is then given back, it remains as a spilled row while later source occurrences stay plain. Both cases are checked controls.

`RelationTransferCoordinates` converts the successful mask into strictly increasing, bounded original logical positions. Independent deletion by those positions equals the mask specification. If give-back appends rows, the original positions leave that suffix untouched: their bounds refer to the original prefix, not a newly shortened list.

A second worker marks a physical live-mask, advancing its logical coordinate only at a live row. Decoding its result equals logical deletion; settling the marks leaves that readout unchanged. Composing these laws with transfer proves full bag preservation for the physical-mask representation and the give-back suffix. For physical rows $[99, 7, 98, 9, 7]$ with live mask $[0, 1, 0, 1, 1]$, removing logical position one removes 9, not the physical row at position one. Controls distinguish that error and keep repeated 7s as separate occurrences.

Native mark selection and array writes must realize these workers. The one-relation framing law does not itself verify every mutable multi-head pass or every native index update.

8 Publication and captured readers

Planning is private. Recoverable preparation refusal leaves the public store unchanged, while successful publication installs the complete plan. The copy-before-delete interval is not a public semantic state: publishing one newly copied 7 while retaining both old copies would temporarily expose three occurrences instead of two. A checked negative control exhibits that difference.

`RelationTransferPublication` makes that interval executable in the model. A draft holds processed original rows, their acceptance mask, pending original rows and the target. Each step offers one pending row and appends its actual disposition. The compensated logical store includes only refused processed rows, all pending rows and the target. Its bag is invariant after

each step, even though the physical store still contains accepted source copies. Any finite page preserves the invariant, and

$$\mathsf{advance}(a + b, d) = \mathsf{advance}(b, \mathsf{advance}(a, d)).$$

One row is consumed from the worklist per step, so a budget at least the original row count completes the model worker. This is a bound on abstract offers, not on native instruction cost.

A session distinguishes available, working and halted states. A working session returns *busy*, not an empty bag; halted work returns *terminated*, not the old value. Publication accepts only completed work. Every value returned from a consistent session is the original bag. These distinctions specify the required isolation contract; they do not assert that the native take-over loop is currently resumable.

Publication uses the existing detach-before-edit reader protocol. `published_bag_and_reader` proves both that the newly published bag equals the original bag and that the old reader's remaining sequence is unchanged. With the example above, the new physical enumeration is $[7, 9, 7]$. A reader already at position one must retain its old suffix $[7, 9]$; adopting the new revision without capture would instead read $[9, 7]$. The incremental worker's `published_bag_and_captured_sequence` theorem establishes the same pair of observations after any admitted completed session.

Native realization must protect the copy/delete window and enumerate the captured roots. The inspected `cetta_malloc` and `cetta_realloc` abort on allocation failure. A fatal abort after partial physical work is not a recoverable refusal or a rollback certificate. Conversely, raw allocation failures during preparation or relation insertion can follow refusal/give-back routes. The obligation map keeps those dispositions distinct.

8.1 Pinned native failure controls

The r82 executable named in the qualification note was exercised with three source shapes: nested facts in a tuple table, diagonal binary facts in array rows, and binary star facts in an actual bit row. Reviewed call-site probes simulate one NULL allocator result before the allocator runs, retaining the old allocation in the realloc cases. Registers are deliberately changed in these private child processes; this differs from the observation-only route coverage probes. Executable, source and harness identities are checked.

Across the three shapes, thirteen recoverable-failure controls preserve the complete reference observations. They cover initial and growing move storage, the taken-position array, tuple/hold growth and binary hold growth. The latter growth failures force give-back of 64 entries for the tuple/diagonal controls and 63 for the star. The row-kind receipt distinguishes the actual array and bit cases. Three snapshot-detachment allocation failures terminate with SIGABRT. Three deliberate mutants skip source-row removal and are rejected by an answer-bag mismatch. These are nineteen reached controls, not exhaustive allocation-failure or concurrent-interleaving coverage. Fatal termination does not establish rollback.

Retained-reader controls additionally keep one or seventeen work handles across source requests, deliver a prefix, remove and replace the source facts, and then finish every old reader. Sixteen combinations of nested or flat facts, reader count, plain or eager relation storage, and disabled or eager reclamation preserve the independent original-bag, post-state and outside-owner oracles. The layout probe follows the frozen executable, not the newer checkout's added lifetime-hold field.

Bounded tracing selects the first collection for each universe, active snapshot-context set and record-cache generation. Across the four enabled collection probes, 70 selected live collections invoke 582 checked snapshot callbacks. Every selected census visits all its registered snapshot

providers, and each callback's exact root sequence agrees with its live fields. This does not claim exhaustive interleaving coverage. The disabled-collection controls provide no positive collection witness. A further fault at provider registry growth reaches the underlying allocator and terminates with SIGABRT; the defensive false-return branch is not credited as exercised.

9 Existence certificates must include compact open facts

The final observer audit found a separate failure in the pinned r82 engine. With only `(seen0 $open)` in an eagerly compacted set, ordinary `(match S (seen0 b) t)` finds an answer. Yet the native empty-collapse test reports absence. A case-based insert-on-absence guard consequently adds an extra ground fact. The direct match and its optimized existence observer disagree; this is not explained by answer ordering.

The reviewed native trace returns false from `space_contains_exact`, true from `space_contains_only_exact_atoms`, and then false with applicability set from `space_match_exists_ground_exact`. The inspected exactness summary accounts for the ordinary row array, which is insufficient when compact storage contains a variable-bearing fact. The reproducer survives the source-variant checks and is reduced to one stored open fact and one ground query.

`MatchExistenceAdmission` reuses the mixed store of `ShapeHeadRouting`. It proves two different permissions. A positive identity hit establishes existence when the query matches itself. A negative hit needs a complete certificate that every matching logical candidate would be that identical fact. An executable whole-store classifier folds both ordinary and compact components and satisfies this contract when its local classifier is sound. Equivalent representations produce the same summary. The query-specific certificate may be more permissive than global groundness; facts that provably cannot match need not prevent the optimization.

A one-column control interprets `none` as a fresh variable and `some n` as a ground atom. An empty ordinary row array incorrectly certifies absence when the compact component contains `none`. The complete summary declines that miss; a present ground fact still allows a positive hit even alongside the variable. A final control shows the wrong absence changing the post-effect bag size from one to two. These are checked permission laws and counterexamples, not an applied C repair.

A later provenance-backed frozen successor candidate keeps the open first fact in ordinary rows when its head subsequently takes a relation. The expanded existence cases then agree, as do the complete joined qualification packages. This is an observed repair in the candidate artifact; the work here neither changed the C source nor treats the reconstructible build receipt as a cryptographic ELF-to-source proof.

9.1 The coordinate helper has a smaller observation domain

The HE empty-case path uses a separate coordinate-form helper behind `space_match_exists_flat_linear_view64`. Its answer is not automatically a whole-space answer. `MatchExistencePartition` proves the required composition:

$$\mathsf{exists}_{R+C}(q) = \mathsf{exists}_{C}(q) \vee \mathsf{exists}_{R}(q),$$

where C is compact storage and R ordinary rows. A compact hit finishes the Boolean observation. A certified compact miss licenses a row-only answer. If compact matching is undecided, materializing the relevant head preserves the whole bag and leaves no matching compact fact, after which rows are the complete domain. Unrelated compact heads may remain stored.

Optional evidence keeps unknown distinct from false. The merge accepts any positive witness, returns false only from two negative domain certificates, and otherwise remains unknown. The model proves that an admitted merged answer is correct. It also records the asymmetric mutation laws for a fixed query: a positive witness survives additions, while absence survives restrictions. Arbitrary insertion can invalidate absence and arbitrary removal can invalidate presence.

Pinned r82 and successor-candidate controls exercise twelve HE scenarios: ground, open and unrelated-open stores; plain and eager storage; direct and explicitly materialized observation. Direct traces distinguish compact hit, compact miss followed by rows, and settle followed by rows. Materialized controls take no specialized view-existence call and supply the same fixed answers. Eight negative harness controls reject cross-call, missing, late, refused and incomplete evidence. These sequential caller-chain receipts do not establish concurrent stability or source-to-binary correspondence.

10 The semantic quantities entering the cost models

`RelationOwnershipCostBridge` partitions the actual acquire receipt into IDs absent from the dictionary and dead IDs already in it. Their total is bounded by row arity; revived IDs are bounded by the dictionary's dead-ID set. These facts make the insertion clamps in `RelationNumberingCost` identities when its dead-count field denotes that set. Release receipts are bounded by both row arity and prior live ownership.

For transfer, successful moves are bounded by independently counted eligible store candidates. If the pending estimate covers those candidates, the movement clamp in `RelationTransferCost.pass` is also an identity. The remaining-row count agrees with subtracting the actual successful mask count. The current imported cost model includes two row scans and the moves, allowing five units per counted promotion row in its abstract work bound.

These are local interfaces between independently proved semantics and cost accounting. Full native scheduling, room growth, failed reservations, mark probes and machine time need their own trace and representation maps.

11 Implementation obligations

Runtime seam	Checked semantic contract	Remaining native obligation
`dense_hold`, `number_hold`	Word/count refinement, receipts and binary use bound	Preserve premises; enforce tuple bounds; execute callbacks
`dense_unhold`, `number_unhold`	Safe decrements and exact release	Live-entry/index correspondence
`dense_number_or_add`, `number_of`	ID reservation and current lookup	Array realization and failed reservation metadata
`dense_renumber`, `tuples_renumber`	Stable lookup, counts, sortedness and side candidates	Hash/side rebuilds and in-place bit writes
`space_dense_list` and snapshot root callbacks	Root enumeration, metadata leases and scoped census	Complete registration, native leases and synchronized updates
Provider add/remove	Coherent indexed census, shared roots and swap-removal	Callback/context encoding and complete enrollment
Exact existence	Whole-domain and partitioned negative-certificate laws	Include compact non-exact facts or retain the caller's compact-domain certificate
`space_dense_take_over_rows`	Stepwise bag conservation and original-coordinate deletion	Eligibility, physical realization and isolation
`space_dense_settle_head`	Give-back preserves the target bag	Exact skeleton reconstruction and ownership handoff
Pinned readers and removal marks	Bag, logical/physical mark bridge and retained readout	Pointer lifetime, detachment and array mutation

The eighteen mathematical modules and `RelationCorrespondenceAudit` pass the targeted build and strict warning-as-error checks. Declaration manifests cover 1,216 checked bodies, with nineteen nonlogical recursion companions listed separately. There are no admissions or added axioms; primitive dependencies are confined to `propext`, `Classical.choice` and `Quot.sound`. This is a model-level correspondence package and obligation map, not a whole-C verification or a claim of information-minimal storage.